

ELEC 278 Social Media Assignment Report

Alex Lester

Queen's University

ELEC 278

Professor Thomas Dean

Tuesday, December 3rd, 2024

Table of Contents

1. Comparison of Potential Data Structures	1
2. User Profiles Management	1
2.1 create_user()	2
2.2 delete_user()	3
2.3 print_users()	3
2.4 change_user_email() & change_user_name()	3
2.5 search_user_by_email() & search_user_by_name()	3
3. Friends	3
3.1 add_friend()	4
3.2 delete_friend()	4
3.3 print_friends()	4
3.4 mutual_friends()	4
3.5 print_mutual_friends()	4
4. Posts	5
4.1 new_post()	5
4.2 add_like()	5
4.3 display_feed()	5
5. Messages	6
5.1 create_message()	6
5.2 print_message()	7
5.3 display_chat()	7
References	8

List of Figures

Figure 1: Image of User Struct From Code	2
Figure 2: Image of HashTable Struct From Code	2
Figure 3: Image of the Chat Struct	6

List of Tables

Table 1: Pros and Cons Table Comparing Different Data Structures	1
--	---

1. Comparison of Potential Data Structures

Table 1 below outlines key data structures that I considered for each part of the assignment. The table highlights the pros and cons associated with each data structure and I reference this table during my rationale for why a certain data structure was determined.

Table 1: Pros and Cons Table Comparing Different Data Structures

Data Structure	Pros	Cons
Hash Table	- Fast average-case lookups, insertions, and deletions ($O(1)$) - Useful for implementing structs	- Requires a good hash function to minimize collisions - Requires a lot of memory
Linked List	- Dynamic size, efficient memory usage - Fast insertion and deletion from the beginning of the list	- Average search cases is $O(n)$ - Requires extra memory for node pointers
Binary Search Tree	- Sorted structure, enabling efficient lookups, insertions, and deletions ($O(\log n)$ for balanced trees)	- If not balanced, the time complexity of operations like search, insertion, and deletion degrades to $O(n)$ in the worst case - Requires additional memory for storing parent, left, and right child pointers in each node.

2. User Profiles Management

For the user management I opted to utilize a closed hash table to store the users. The reason for this is that a hash table offers fast average-time complexity $O(1)$ for insertions, deletions and lookups by key. The hash table is stored as a global variable and its array size is set to 10007 as the system is meant to accommodate up to 10000 users. By setting the table size to be a prime number it will minimize the number of collisions. A two hash table system was implemented, one where the email is hashed and one where the users name is hashed. Although hash tables require more memory due to their size and the potential for unused slots, this trade-off is acceptable given the platform's need for quick and consistent performance. Other data structures, like linked lists or binary search trees, would have resulted in higher average operation times ($O(n)$ for linked lists or $O(\log n)$ for binary search trees), making them less suitable for this use case where speed is a priority.

For the user struct, I implemented the following: an integer ID, a string for the name, a string for the email, a linked list of friends, an integer to track the number of friends, a linked list of posts, an integer to track the number of posts, and a linked list for chats. Refer to Figure 1 to see an image of the User Struct. The hash table data structure will be adapted for this user struct by creating a HashTable Struct. In Figure 2 it shows that this struct has an array of the User struct pointers called table which is the size of 10007. It also contains an integer variable called size which stores the number of spots that have been filled in the table.

```
typedef struct {
    // add attributes
    int id; // int id field
    char name[50]; // string for name with 49 characters + 1 for null character
    char email[50]; // string for email with 49 characters + 1 for null character
    struct FriendNode* friends; // Pointer to the linked list of friends
    int numFriends; // int number of friends field
    struct PostNode* posts; // pointer to linked lists of posts made by the user
    int numPosts; // int number of posts field
    struct ChatNode* chats; // pointer to linked lists of chats made by the user
} User;
```

Figure 1: Image of User Struct From Code

```
typedef struct {
    User* table[10007]; // Array to store pointers to User objects
    int size; // int number of users currently stored in the hash table
} HashTable;
```

Figure 2: Image of HashTable Struct From Code

2.1 create_user()

The create_user() function allocates memory to create a new user pointer. The name and email strings passed in the parameter are stored in this pointer, as well as the friends and post pointers are set to NULL. The numFriends and numPosts field are set to zero and the id field is set to the global variable numUsers before incrementing it by one giving it a unique ID tag. The function then checks that the name and email are less than 50 characters (1 character for the NULL character) and calls the search_user_by_email() and search_user_by_name() functions. If they both return NULL indicating that it is a unique name and email the function calls user_insert() which is a helper function and then will return the user pointer. In user_insert() it will call another helper function called string_hash() which utilizes the djb2 hash function created by Dan Bernstein [1]. It returns a hash and the function will linearly probe until it finds a spot in the hash table where it is NULL where it will then set that to be the user pointer. This function has an average time complexity of $O(1)$ however in the worst cases $O(n)$ with n being the size of the table.

2.2 delete_user()

In delete_user() it will call on a helper function called locate_user() which retrieves the hash using string_hash() and will linearly prob until it finds the user. It returns the index of where the user was found and sets the place in the hash table to be NULL and frees the user pointer. The average time complexity is $O(1)$.

2.3 print_users()

In print_users it creates an array with the size of the total number of users in the hash table. It will then search through the hash table adding each users name to this array. This has a time complexity of $O(\text{Table_Size})$. Qsort is called which is a built in c function which sorts the names array. This has an average time complexity of $O(n \log n)$ and the names array is then iterated through printing each name, $O(n)$. Thus this function has an average time complexity of $O(\text{TABLE_SIZE} + n \log n)$ However since $n \log n < n \leq \text{TABLE_SIZE}$ and the complexity becomes $O(\text{TABLE_SIZE})$.

2.4 change_user_email() & change_user_name()

Both of the two functions operate the exact same. This function calls the function locate_user() it then at the index in the hash table that it returns sets it to NULL. It then changes the users email or name to the new email or name and calls user_insert(). The average time complexity is $O(1)$ as searching and deleting and inserting users is a constant time complexity.

2.5 search_user_by_email() & search_user_by_name()

Both of the two functions operate in the same manner. It calls string_hash() function to get the index of the email or name. It then linearly probs through the hash table until it finds the user. If it locates the user it will return it however if it linearly probs through the entire table it will return null thus giving it an average time complexity of $O(1)$ however it has a worst case of $O(n)$ where n is the size of the table.

3. Friends

To manage and store the friends data for different users a linked list data structure was utilized. Referring to Table 1 some pros for linked lists have a constant $O(1)$ insertion and utilises dynamic memory as only the necessary memory is required unlike in a hash table. The most common use for friends in a social media app is adding new friends compared to finding mutual friends or deleting friends. As such by utilizing a data structure that is both memory efficient and constant insertion complexity will ensure the best choice for managing this data. While other data structures are also viable options using a hash table or a BST is not necessary for the most common uses.

The linked list was implemented by creating a FriendNode struct which had the fields next and user. A pointer of FriendNode is initial set to NULL inside the User struct, refer to Figure 1 and each time a friend is added it is appended to that linked list in the User. The pointer within the User is a pointer to the head of the linked list and each time a new friend is added they are added at the head.

3.1 add_friend()

The function checks if both users are not the same and if the users are not already friends by calling the helper function check_friends(). In this function goes through one users friends linked list and checks if it the other user is present in the linked list and returns either 0 or 1. If the two users are not friends it will create new FriendNodes for each set the user field to the corresponding user. It will then append the each user to the head of their friends linked list and increment their numFriends field by 1. This function has an average time complexity of $O(n)$, n being the number of friends for one of the users.

3.2 delete_friend()

This function iterates through one users friends linked list and locates the position of the other user in the linked list. It removes it from the linked list and then repeats the same process for the other user. This function has an average time complexity of $O(n)$ n being the length of the friends linked list.

3.3 print_friends()

This functions creates an array with the size of numFriends for the user. It then iterates through the friends linked list and appends the name of each of the users friends. It will then call qsort to sort the array and then call a for loop to print the names. This function has a time complexity of $O(n)$, n being the number of friends of the user.

3.4 mutual_friends()

This function operates by creating a user pointer array and setting two pointers to the head of both user's friends linked list. It then iterates using a nested while loop through both users friends checking if any are the same. If yes it will append the user to the array and utilizing realloc makes the array bigger. After the while loops are finished running it will return the array of users. This function has a runtime complexity of $O(n*m)$ n being the length of one users friends linked list and m being the length of the other.

3.5 print_mutual_friends()

This function takes in an array of user pointers and creates an array of names. It will then add all of the names into the array and call qsort() to sort the array. It then prints out the array. The average time complexity for this function is $O(n \log n)$ as the qsort() function has an average time complexity of $O(n \log n)$.

4. Posts

The posts data structure was implemented the same as the friends data structure. Both of which utilize a linked list stored inside the user struct. To achieve this a PostNode struct was created with the fields post and next which is extremely similar to the FriendNode struct. The reason for creating posts in such a manner is that research is shown that only around 11% of tweets are deleted after being posted on twitter [2]. This shows a roughly 9:1 ratio for creating posts to deleting them. Referring to Table 1 which compares different data structures since linked lists have an insertion time complexity of $O(1)$ it seemed like a better choice than a BST which has $O(\log n)$. Linked list also seemed as a better choice than hash tables as hash tables require a lot of memory to be effective and since it is unclear how many posts a user will make and the fact that this system can sustain 10000 members it seemed that linked list was the better choice for their memory.

The post struct is comprised of an integer id tag, a User pointer, an integer number of like field, a string content field and a FriendNode linked list for which users already like the post.

4.1 new_post()

This function operates by allocating memory for a new post pointer and setting the content and user to the ones passed in the parameters. It will set the id to the global variable numPosts and then increment it by one and it will initialize numLikes and whoLiked by setting it to 0 and NULL. The time complexity for this function is $O(1)$ as it takes a constant time.

4.2 add_like()

This function works by first checking if the liking user is the same as the author of the post or if they are a friend of the author by calling the check_friends() helper function. If it passes the if statement it will then check that the user has not already liked the post by iterating through the post's whoLiked linked list. If the user has not liked it, it will then create a new friend node and add the liking user to it before appending it to the head of the whoLiked linked list associated with the post. It will also increment the numLikes field by one. This function has an average time complexity of $O(n)$ with n being the number of users that liked a post.

4.3 display_feed()

This function works by adding the posts created by the user to an array and iterating through the user's friends linked list and adding their posts to the array. A merge_sort() helper function is then called to sort the array. Merge sort has an average time complexity of $O(n \log n)$ which is why I opted to use it compared to bubble sort, selection sort or insertion sort which have average time complexities of $O(n^2)$. For this function I adapted the merge sort function found online [3] to work with the Post struct and the specific parameters outlined in the assignment of prioritizing posts by number of likes and then recency. The now sorted posts array is printed out the twenty post at the beginning of the array. This function has an average time complexity of $O(n \log n)$ due to merge sort being a dominating factor. In this case n is equal to the amount of posts created by the user and the user's friends.

5. Messages

The messages functionality of this social media platform involves utilizing a both a queue and a linked list. A Message struct was created, along with a Chat struct and a ChatNode struct. The data structure followed a similar data structure for both friends and posts as a pointer for a ChatNode struct called chats it implemented within the User Struct. The ChatNode struct has two elements, a Chat struct pointer and a ChatNode pointer, next. In the Chat struct shown in (Figure 3) has two user pointers, user1 and user2 which point to the two users within the chat. A Messages struct pointer array and with the implementation of an integer front and count variables operates as a queue. In the Messages struct there are the unique integer id tag as well as two User struct pointers to the sender and receiver. It also contains a string that has the content for the message.

```
typedef struct {
    User* user1;           // Pointer to User 1
    User* user2;           // Pointer to User 2
    Message* messages[50]; // Fixed-size array to hold the 50 most recent messages
    int front;             // Index of the oldest message
    int count;             // Number of messages in the queue
} Chat;
```

Figure 3: Image of the Chat Struct

The rationale for utilizing a linked list is due to its memory efficiency. A unique feature of the linked list is that it is able to utilize only the necessary amount of memory it requires to store all of the information. Compared to a hash table which will have open spots and requires a lot of extra memory to avoid collisions. A linked list was also chosen over a BST as even though searching a BST has a quicker time complexity of $O(\log n)$ compared to $O(n)$. It is expected that the number of chats for a given user will not be large and as such it was not necessary to implement a balanced BST.

The reason a queue was implemented is due to the requirements outlined in the assignment description. For a given conversation between two friends only the 50 most recent messages are saved. Since a queue operates on a first in first out (FIFO) system this made it the perfect choice compared to a stack which operates on last in first out (LIFO).

5.1 create_message()

This function works by first checking if the two users are friends by calling the check_friends() helper function. If yes it will check if a chat already exists between the two users by iterating through the chats linked list in the User struct. If not, it will create a new ChatNode and initialize all of the information. It will then create a new message pointer set its information and call the helper function enqueue() to append the message to the queue. The function will then append the ChatNode to the head of both users chats linked list. If the users already have a chat it will check if there are 50 messages in the queue. If yes it will utilize the dequeue() helper function to remove an item from the queue before calling enqueue(). If not it will just call enqueue(). This function has an average time complexity of $O(n)$ where n is the length of the chats linked list in a user.

5.2 print_message()

This function prints the content of the message passed in its parameters and has a constant time complexity of $O(1)$.

5.3 display_chat()

The display_chat() function operates by calling check_friends() to check if the two users are friends. It then iterates through one of the users chats linked list until it finds the chat between the two users. It will then print out the messages found in the queue. This function has an average time complexity of $O(n)$ with n being the length of the chats linked list. This is assuming however that $n \geq m$ with m being the number of messages in the queue. If $n < m$ the time complexity would be $O(m)$.

References

- [1] "Hash Functions," Feb 1990. [Online]. Available:
<http://www.cse.yorku.ca/~oz/hash.html#:~:text=If%20you%20just%20want%20to,K%26R%5B1%5D%2C%20etc.>
- [2] P. Bhattacharya, "Analyzing Regrettable Communications on Twitter: Characterizing Deleted Tweets and Their Authors," December 2022. [Online]. Available:
https://arxiv.org/abs/2212.12594?utm_source=chatgpt.com.
- [3] "C Program for Merge Sort," Geeks For Geeks, October 2024. [Online]. Available:
<https://www.geeksforgeeks.org/c-program-for-merge-sort/>.